Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 1

Aim

Implement Linear and Logistic Regression on real-world datasets.

Linear Regression

Dataset Source

- Dataset: Weight-Height Dataset
- Kaggle: https://www.kaggle.com/datasets/mustafaali96/weight-height

---

Dataset Description

The Weight-Height dataset is a widely used bivariate dataset designed for introductory regression tasks in machine learning. It contains 10,000 records describing individuals with three attributes: Gender, Height (in inches), and Weight (in pounds). The relationship between height and weight in this dataset is almost perfectly linear with very little noise, making it ideal for demonstrating and validating Simple Linear Regression models. The data is provided as a CSV (Comma Separated Values) file named weight-height.csv, which includes one categorical column (Gender) and two numerical columns (Height and Weight) suitable for modeling and visualization.

Mathematical Formulation of the Algorithm

Regression analysis studies how a target (dependent) variable changes in response to one or more predictor (independent) variables. When the target is continuous (such as weight, salary, or temperature), regression models are appropriate. Simple Linear Regression (SLR) focuses on a single predictor and assumes a linear relationship between the input and the output.

The general form is:

$$y = mx + c$$

In machine learning notation:

$$y = \beta_0 + \beta_1 x$$

Where:

- y – Output (Weight)
- x – Input (Height)
- β_0 – Intercept
- β_1 – Slope (coefficient)

The goal of Linear Regression is to find the best-fit line that minimizes the error between predicted and actual values. Using Ordinary Least Squares (OLS), the optimal parameters are computed in a single closed-form step by minimizing the sum of squared residuals:

$$\beta_1 = (n\Sigma xy - (\Sigma x)(\Sigma y)) / (n\Sigma x^2 - (\Sigma x)^2)$$
$$\beta_0 = ((\Sigma y)(\Sigma x^2) - (\Sigma x)(\Sigma xy)) / (n\Sigma x^2 - (\Sigma x)^2)$$

where n is the number of data points. OLS guarantees the unique global minimum of the sum of squared residuals, making the solution exact and reproducible without iterative optimization.

Algorithm Limitations

Key limitations of Simple Linear Regression:

1. **Assumption of Linearity**

   It enforces a straight-line relationship and assumes a constant change in weight for each unit increase in height. If the true relationship is curved or non-linear, the model underfits and produces systematically biased predictions.

2. **Sensitivity to Outliers**

   Because OLS squares errors, extreme outliers can heavily influence the regression line, altering slope and intercept to fit a few anomalous points at the expense of overall accuracy.

3. **Heteroscedasticity**

   SLR assumes constant variance of errors (homoscedasticity). If variance grows with height (wider spread at taller heights), this assumption is violated and statistical inferences become unreliable.

4. **Extrapolation Risk**

The linear relationship is valid only within the observed range of heights. Predictions far outside the data range can be unrealistic (e.g., negative weights).

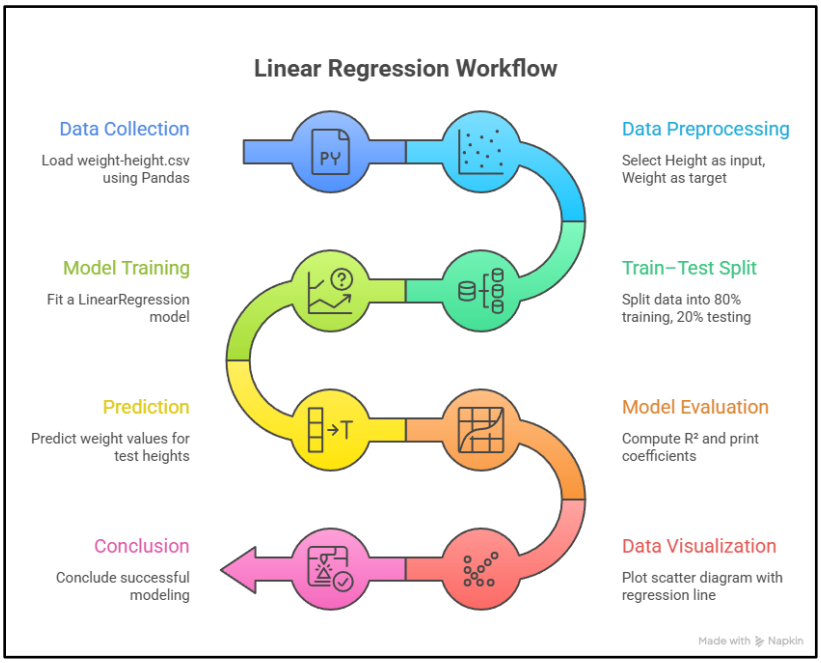
5. Independence of Observations

The method assumes each sample is independent. In time-series or grouped settings, autocorrelation violates this assumption, causing underestimated standard errors.

Methodology / Workflow

1. **Data Collection**
   - Load weight-height.csv using Pandas into a DataFrame containing Gender, Height, and Weight.
2. **Data Preprocessing**
   - Select Height as the input feature X and Weight as the target y.
   - Reshape Height to a 2D array as required by scikit-learn.
   - No missing-value handling is needed since the dataset is clean.
3. **Train–Test Split**
   - Split into 80% training and 20% testing using train_test_split.
   - The training set is used to learn the height–weight relationship; the test set evaluates generalization.
4. **Model Training**
   - Fit a LinearRegression model on the training data so it learns the slope and intercept that minimize squared error.
5. **Prediction**
   - Use the trained model to predict weight values for the test heights.
6. **Model Evaluation**
   - Compute R^2 (coefficient of determination) to measure how well predictions match actual weights.
   - Print the learned coefficient (slope) and intercept, which together define the final regression equation.
7. **Data Visualization**
   - Plot a scatter diagram of actual test data points (Height vs Weight) and overlay the fitted regression line.
   - This visually shows how closely the line follows the data.
8. **Conclusion**
   - Conclude that Simple Linear Regression successfully models the near-linear relationship between height and weight and that the high R^2 and well-aligned line indicate a good fit.
1. **Data Collection**
   - Load student-mat.csv into a Pandas DataFrame.

2. **Data Preprocessing**
   - Identify categorical variables (e.g., school, sex, address, famsize, schoolsup, famsup, internet, romantic).
   - Apply one-hot encoding using get_dummies with drop_first=True to convert categoricals to numerical form.
   - Handle any missing values if present (mean imputation for numeric columns).
3. **Train–Test Split**
   - Split into 80% training and 20% testing using train_test_split with random_state=42.
4. **Model Training**
   - Train a Multiple Linear Regression (OLS) model on the training set.
5. **Prediction**
   - Predict G3 for the test set.
6. **Model Evaluation**
   - Compute RMSE to measure average prediction error in grade points.
   - Compute R^2 to measure the proportion of variance in G3 explained by the predictors.
7. **Data Visualization**
   - Plot a scatter diagram of Actual G3 vs Predicted G3 with an ideal reference line (y = x).
   - Points close to the red dashed line indicate accurate predictions.
8. **Conclusion**
   - If R^2 is high and RMSE is around 2–3 grade points, the model is capturing most of the variation; remaining error suggests scope for regularized models (Lasso and Ridge).



Performance Analysis

The performance of the implemented Simple Linear Regression model on the Weight–Height dataset is evaluated using the R^2 score together with the learned slope and intercept of the regression line. Since OLS-based Linear Regression has no hyperparameters to tune, this analysis represents the complete and final model performance.

1. **R^2 Score (Coefficient of Determination)**

   The obtained $R^2 = 0.8577$ indicates that approximately 85.77% of the variation in weight can be explained by a person's height alone. This shows that height is a very strong predictor of weight in this dataset and that the linear model captures most of the underlying relationship. The remaining 14.23% of the variation is likely due to factors not included in the model, such as body composition, age, gender differences, and lifestyle, as well as natural biological variability. Overall, the model demonstrates a good fit and strong predictive performance for this simple single-predictor setting.

2. **Interpretation of the Slope (Coefficient)**

   The learned slope is 7.7022, meaning that for every 1-inch increase in height, the model predicts an average increase of approximately 7.7 pounds in weight. This confirms a clear positive linear relationship: taller individuals are expected to weigh more, and the magnitude of the slope reflects a relatively steep and consistent dependence of weight on height across the dataset.

3. **Interpretation of the Intercept**

   The intercept is −349.7878, which is the predicted weight when height is mathematically set to zero inches. Although this value has no physical meaning (no person has 0 inches of height), it is necessary for correctly positioning the regression line relative to the observed data points. The intercept should therefore be interpreted as a mathematical model artifact rather than a realistic weight prediction.

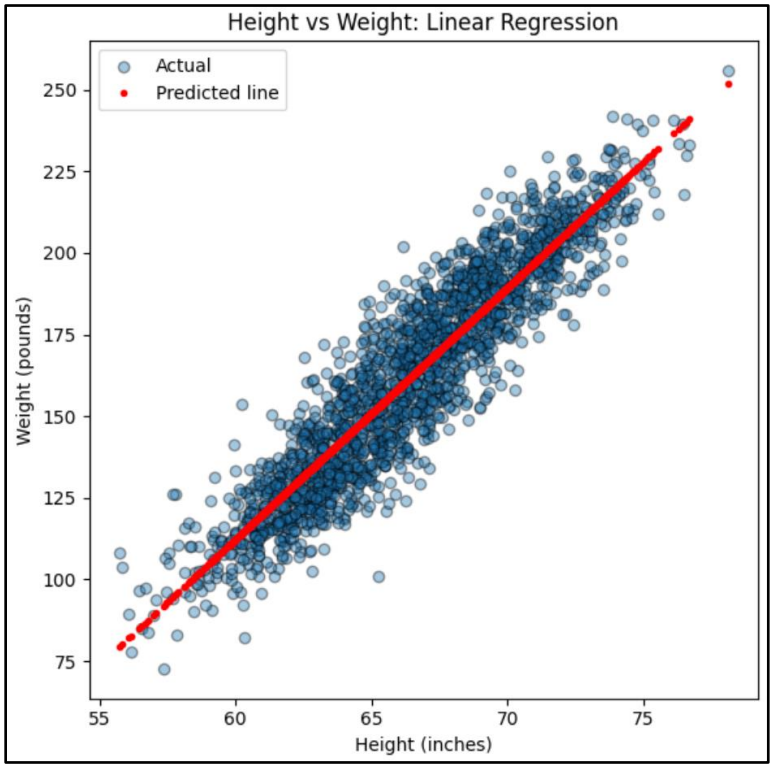
Hyperparameter Tuning

Linear Regression solved via Ordinary Least Squares (OLS) has no tunable hyperparameters. The algorithm directly computes the unique analytically optimal slope and intercept through the closed-form OLS equations, without any regularization constants, learning rates, or penalty terms to configure. There is no iterative optimization process and therefore no parameter search is possible or necessary. Hyperparameter tuning is not applicable for this model.

Code and Output – Linear Regression

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.model_selection import
train_test_split

from sklearn.linear_model import
LinearRegression

from sklearn.metrics import r2_score


# 1. Data Collection

df = pd.read_csv('weight-height.csv')

print("First 5 rows:")

print(df.head())

print("\nShape:", df.shape)


# 2. Data Preprocessing

X = df[['Height']].values   # 2D array

y = df['Weight'].values


# 3. Train–Test Split (80/20)

X_train, X_test, y_train, y_test =
train_test_split(

    X, y, test_size=0.2, random_state=42

)


# 4. Model Training

lin_reg = LinearRegression()

lin_reg.fit(X_train, y_train)


# 5. Prediction

y_pred = lin_reg.predict(X_test)


# 6. Evaluation

r2 = r2_score(y_test, y_pred)

slope = lin_reg.coef_[0]

intercept = lin_reg.intercept_

print("\n--- Linear Regression Results ---
")

print(f"R²:        {r2:.4f}")

print(f"Slope:     {slope:.4f}")

print(f"Intercept: {intercept:.4f}")


# 7. Visualization

plt.figure(figsize=(6, 6))

plt.scatter(X_test, y_test, alpha=0.4,
label='Actual', edgecolor='k')

plt.plot(X_test, y_pred, 'r.',
label='Predicted line')

plt.xlabel('Height (inches)')

plt.ylabel('Weight (pounds)')

plt.title('Height vs Weight: Linear
Regression')

plt.legend()

plt.tight_layout()
```

```
plt.show()
```



Logistic Regression

Dataset Source

- Dataset: Advertising Dataset
- Kaggle: https://www.kaggle.com/datasets/gabrielsantello/advertisement-click-on-ad

---

Dataset Description

The Advertising dataset is a multivariate dataset commonly used for binary classification tasks such as Logistic Regression. It contains 1,000 observations describing web users' demographic characteristics and online browsing behavior, with the objective of predicting whether a user will click on an online advertisement.

The dataset includes a mix of numerical and categorical variables. The numerical features used as predictors are:

- Daily Time Spent on Site (minutes)
- Age (years)

- Area Income (average income of the user's geographical area)
- Daily Internet Usage (minutes)
- Male (binary indicator: 1 for male, 0 for female)

The target variable is Clicked on Ad (binary: 1 = clicked, 0 = did not click). Text and timestamp columns (Ad Topic Line, City, Country, Timestamp) are dropped during preprocessing as they are non-numeric and not directly informative for the model. The dataset is well-balanced with 500 instances of each class across 1,000 records.

Mathematical Formulation of the Algorithm

Logistic Regression is a supervised learning algorithm used for binary classification (e.g., Click / No Click). Unlike Linear Regression which predicts continuous values, Logistic Regression predicts a probability between 0 and 1 that an input belongs to the positive class.

The model first computes a linear combination of the input features:

$$z = b_0 + b_1 x_1 + b_2 x_2 + \ldots + b_n x_n$$

This linear score z is then passed through the Sigmoid (logistic) function to produce a probability:

$$\hat{y} = 1 / (1 + e^{-z})$$

Where:

- $\hat{y}$ – Estimated probability that the user clicked the ad
- z – Linear score computed from input features and learned weights
- b_0 – Bias (intercept) term
- b_1, b_2, …, b_n – Learned weight coefficients for each feature

Properties of the sigmoid output:

- Large positive z → $\hat{y} \approx 1$ (high probability of clicking)
- Large negative z → $\hat{y} \approx 0$ (low probability of clicking)
- z = 0 → $\hat{y} = 0.5$ (maximum uncertainty; classification threshold)

The model parameters are estimated by maximizing the log-likelihood via the binary cross-entropy loss function:

$$L = -(1/n) \Sigma [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

A threshold of 0.5 is applied to the output probability to assign class labels: if $\hat{y} \geq 0.5$, predict class 1 (Clicked); otherwise predict class 0 (Did not Click). The S-shaped sigmoid curve maps any real-valued input to the interval (0, 1), making it ideal for producing interpretable class probabilities.

Algorithm Limitations

Key limitations of Logistic Regression:

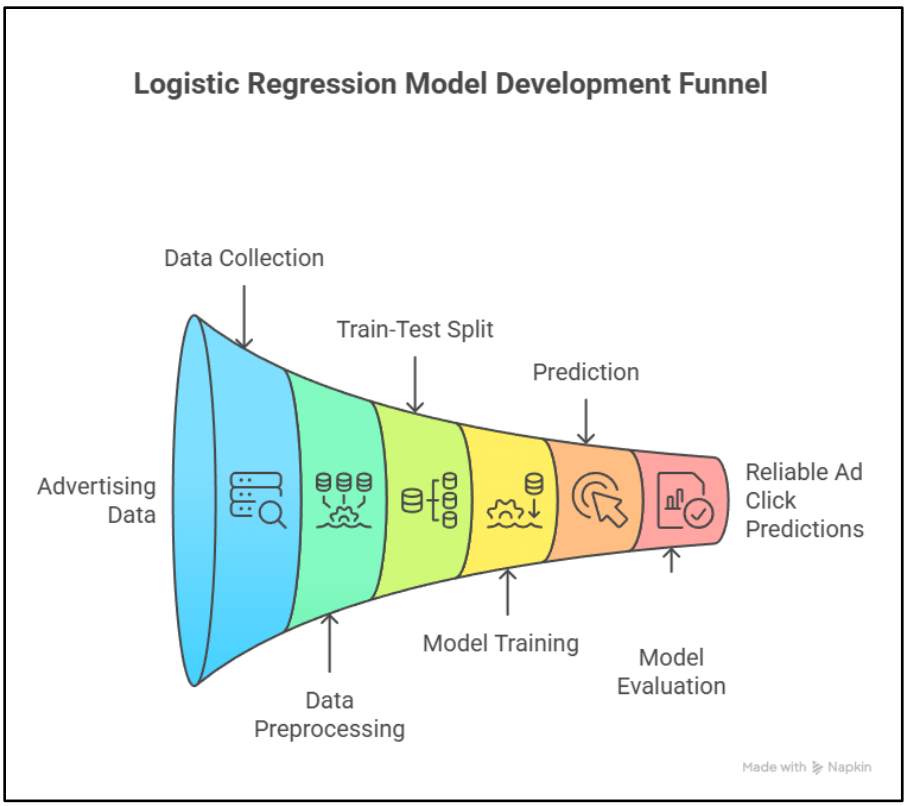
1. **Linear Decision Boundary**
   Logistic Regression assumes that classes can be separated by a linear hyperplane in the feature space. It struggles with complex non-linear class boundaries (e.g., circular or spiral separation), and requires feature engineering or kernel methods in such cases.
2. **Sensitivity to Multicollinearity**
   Highly correlated features lead to unstable and unreliable coefficient estimates, making the interpretation of individual predictors difficult, even when overall predictive accuracy remains high.
3. **Impact of Outliers**
   Extreme data points can strongly shift the decision boundary as the model tries to minimize their loss, potentially reducing accuracy on the majority of normal data.
4. **Need for Adequate Sample Size**
   Parameters are estimated by Maximum Likelihood Estimation (MLE). With very small datasets, coefficient estimates can be biased or numerically unstable. A common rule of thumb is at least ~10 observations per predictor variable.
5. **Class Imbalance**
   When one class strongly dominates (e.g., 99% no-click), the model may predict the majority class for all inputs and still achieve high accuracy, masking poor performance on the minority class. Techniques such as class weighting or resampling are required in such situations.

Methodology / Workflow

1. **Data Collection**
   - Load advertising.csv into a Pandas DataFrame and inspect shape and sample rows.
2. **Data Preprocessing**
   - Drop non-numeric columns: Ad Topic Line, City, Country, Timestamp.

- Keep numeric predictors: Daily Time Spent on Site, Age, Area Income, Daily Internet Usage, Male.
- Set Clicked on Ad as the target variable y.

3. **Train–Test Split**
   - Split into 70% training and 30% testing using train_test_split with random_state=101 to ensure reproducibility.
4. **Model Training**
   - Without Tuning: Fit a default LogisticRegression model (C=1.0, penalty='l2', solver='lbfgs') directly on the training data.
   - With Tuning: Define a parameter grid over C (0.001 to 1000) and penalty (l1, l2) using the liblinear solver. Apply GridSearchCV with 5-fold cross-validation to evaluate 14 parameter combinations (70 total fits) and select the best estimator.
5. **Prediction**
   - Use the trained model (or best GridSearchCV estimator) to predict class labels on X_test. Probabilities are thresholded at 0.5 to produce binary predictions.
6. **Model Evaluation**
   - Compute Accuracy on the test set.
   - Generate a Classification Report (precision, recall, F1-score) for both classes.
   - Compute and visualize a Confusion Matrix (TP, TN, FP, FN).
7. **Data Visualization**
   - Plot the confusion matrix as a Seaborn heatmap to visually inspect where the model makes correct predictions and where it makes mistakes (missed clicks vs. false alarms).
8. **Conclusion**
   - Summarize that Logistic Regression reliably predicts ad clicks from user browsing and demographic features, and that hyperparameter tuning via GridSearchCV provides statistical confidence in the final model configuration.



Performance Analysis

The performance of the Logistic Regression model on the Advertising dataset is evaluated using the Accuracy Score, Classification Report, and Confusion Matrix. The analysis below reflects the model trained with default parameters (without hyperparameter tuning) — LogisticRegression with C=1.0, penalty='l2', and solver='lbfgs'.

1. **Accuracy Score**

   The default model achieves an accuracy of 97.67% on the test set, correctly classifying 293 out of 300 samples. This indicates that the selected features — Daily Time Spent on Site, Age, Area Income, Daily Internet Usage, and the Male indicator — are highly informative for predicting ad click behaviour. The small error rate of approximately 2.33% demonstrates that the classifier generalizes very well to unseen data even without any hyperparameter tuning.

2. **Confusion Matrix Analysis**

   The confusion matrix obtained is [[155, 2], [5, 138]]:

   - True Negatives (155): The model correctly predicted "Did not Click" for 155 users who actually did not click the ad.

- False Positives (2): Only 2 users were incorrectly predicted as “Click” when they actually did not click, representing a very low false alarm rate.
- False Negatives (5): The model missed 5 users who actually clicked but were predicted as “No Click”, indicating a small number of missed opportunities.
- True Positives (138): The model correctly predicted “Click” for 138 actual clickers.

The dominance of the diagonal entries (155 and 138) and the very small off-diagonal entries (2 and 5) demonstrate that the model distinguishes between the two classes with high reliability.

3. Classification Report

For the negative class (0 – Did not Click):

- Precision: 0.97 → When the model predicts “No Click”, it is correct 97% of the time.
- Recall: 0.99 → It correctly identifies 99% of all actual non-clickers.
- F1-score: 0.98 → Strong balance between precision and recall for this class.

For the positive class (1 – Clicked on Ad):

- Precision: 0.99 → When the model predicts “Click”, it is correct 99% of the time, meaning very few wasted marketing efforts on non-interested users.
- Recall: 0.97 → It successfully captures 97% of all users who actually clicked, missing only a small fraction of potential customers.
- F1-score: 0.98 → Excellent trade-off between catching true clickers and avoiding false alarms.

The macro and weighted averages (precision, recall, F1 ≈ 0.98) confirm that performance is consistently high for both classes, with no significant imbalance in how well each class is handled.

Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV with 5-fold cross-validation to systematically identify the optimal regularization configuration for the Logistic Regression model.

1. Baseline Performance (Before Tuning)

The default LogisticRegression model used C=1.0, penalty='l2', and solver='lbfgs'. On the test set it achieved an accuracy of 97.67%, a confusion matrix of [[155, 2], [5, 138]], and an F1-score of 0.98 for both classes. While this is already a high-performing result, it does not provide statistical confidence that the chosen defaults are the best possible configuration.

2. Search Strategy

A grid search was conducted over the following parameter space:

- C (inverse regularization strength): [0.001, 0.01, 0.1, 1, 10, 100, 1000]
- penalty: ['l1', 'l2']
- solver: ['liblinear'] — selected as it supports both L1 and L2 penalties and is efficient for small to medium datasets

This produced 14 unique parameter combinations. With 5-fold cross-validation, a total of 70 model fits were evaluated. The combination yielding the highest mean cross-validated accuracy was selected as the best estimator.

3. Best Hyperparameters Identified

- C = 100
- penalty = 'l1'
- solver = 'liblinear'

4. Performance After Tuning

The tuned model achieved an accuracy of 97.67% on the test set, with the same confusion matrix [[155, 2], [5, 138]] and identical classification report metrics (precision, recall, F1 ≈ 0.98 for both classes) as the default model.

5. Impact of Tuning

The accuracy remained at 97.67% before and after tuning. This outcome is meaningful and not a failure of the tuning process — it demonstrates that the Advertising dataset's features are sufficiently strong and linearly separable that even a default configuration captures the decision boundary well. The tuning process is still significant for the following reasons:

- Statistical Confidence: The cross-validated search confirms that 97.67% is a robust and reproducible result, not a product of a favourable test-set split. The exhaustive grid search rules out the possibility of a better configuration existing within the defined parameter space.
- Regularization Insight: The shift from the default C=1 to the optimal C=100 reveals that weaker regularization is preferred. This means the features are reliable and strong predictors that benefit from having their coefficients less constrained.
- Sparsity via L1: The shift from the default L2 penalty to L1 confirms that some features contribute more than others. L1 regularization drives less

important coefficients exactly to zero, effectively performing automatic feature selection and producing a more interpretable model.

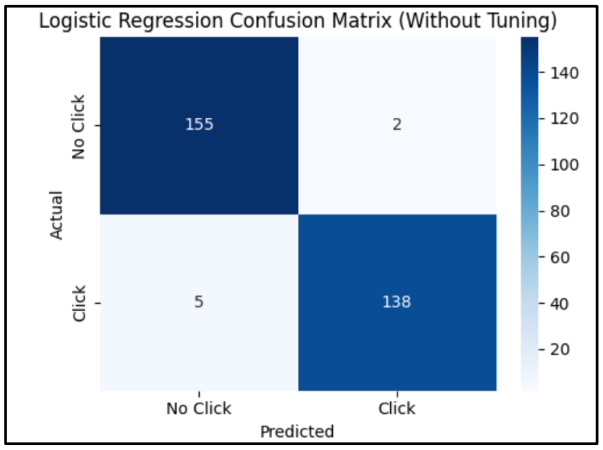
- Eliminates Doubt: The systematic search provides statistical justification that the chosen configuration is optimal within the grid, making the final model the most well-validated and reliable choice for deployment.

Code and Output – Without Hyperparameter Tuning

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# 1. Data Collection

df = pd.read_csv('advertising.csv')

print("First 5 rows:")

print(df.head())

print("\nShape:", df.shape)


# 2. Data Preprocessing

drop_cols = ['Ad Topic Line', 'City', 'Country', 'Timestamp']

df = df.drop(columns=drop_cols)

X = df.drop(columns=['Clicked on Ad'])

y = df['Clicked on Ad']


# 3. Train-Test Split (70/30)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=101

)


# 4. Model Training (Default Parameters)

log_reg = LogisticRegression(max_iter=1000)

log_reg.fit(X_train, y_train)


# 5. Prediction

y_pred = log_reg.predict(X_test)


# 6. Evaluation

acc = accuracy_score(y_test, y_pred)

print(f"\nAccuracy: {acc*100:.2f}%\n")

print("Classification Report:\n")

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:\n", cm)

# 7. Visualization
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Click', 'Click'],
            yticklabels=['No Click', 'Click'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Logistic Regression Confusion Matrix (Without Tuning)')
plt.tight_layout()
plt.show()
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.97      0.99      0.98       157
           1       0.99      0.97      0.98       143

    accuracy                           0.98       300
   macro avg       0.98      0.98      0.98       300
weighted avg       0.98      0.98      0.98       300

Confusion Matrix:
 [[155   2]
 [  5 138]]
```

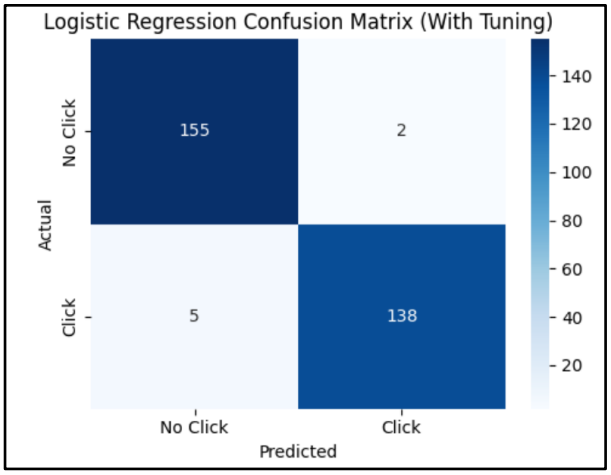


Code and Output – With Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Data Collection
df = pd.read_csv('advertising.csv')
print("First 5 rows:")
print(df.head())
print("\nShape:", df.shape)

# 2. Data Preprocessing
drop_cols = ['Ad Topic Line', 'City', 'Country', 'Timestamp']
df = df.drop(columns=drop_cols)
X = df.drop(columns=['Clicked on Ad'])
y = df['Clicked on Ad']

# 3. Train-Test Split (70/30)
```

```
X_train, X_test, y_train, y_test =
train_test_split(

    X, y, test_size=0.3,
random_state=101

)


# 4. Hyperparameter Tuning with
GridSearchCV

param_grid = {

    'C': [0.001, 0.01, 0.1, 1, 10, 100,
1000],

    'penalty': ['l1', 'l2'],

    'solver': ['liblinear']

}


log_reg =
LogisticRegression(max_iter=1000)

grid = GridSearchCV(

    estimator=log_reg,

    param_grid=param_grid,

    cv=5,

    scoring='accuracy',

    n_jobs=-1

)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_

print("\nBest Parameters:",
grid.best_params_)


# 5. Prediction

y_pred = best_model.predict(X_test)


# 6. Evaluation

acc = accuracy_score(y_test, y_pred)

print(f"\nAccuracy: {acc*100:.2f}%\n")

print("Classification Report:\n")

print(classification_report(y_test,
y_pred))

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:\n", cm)


# 7. Visualization

plt.figure(figsize=(5, 4))

sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',

            xticklabels=['No Click', 'Click'],

            yticklabels=['No Click', 'Click'])

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.title('Logistic Regression Confusion
Matrix (With Tuning)')

plt.tight_layout()

plt.show()
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.97      0.99      0.98       157
           1       0.99      0.97      0.98       143

    accuracy                           0.98       300
   macro avg       0.98      0.98      0.98       300
weighted avg       0.98      0.98      0.98       300

Confusion Matrix:
 [[155   2]
 [  5 138]]
```



Conclusion

The overall practical successfully demonstrates how Linear Regression and Logistic Regression can be applied to real-world datasets for regression and classification tasks respectively. Using the Weight–Height dataset, Simple Linear Regression accurately models the strong linear relationship between height and weight, achieving a high R^2 score of 0.8577 and a clear interpretable regression equation that confirms height as a powerful predictor of weight. For Logistic Regression applied to the Advertising dataset, both the default and the GridSearchCV-tuned model achieved an accuracy of 97.67%, with excellent precision (0.97–0.99), recall (0.97–0.99), and F1-scores (0.98) across both classes, along with a near-perfect confusion matrix. The tuning process — while not improving raw accuracy — provided cross-validated statistical confidence that the model is optimally configured, and revealed meaningful insights: weaker regularization (C=100) and L1 penalty are preferred, indicating strong predictive features and natural sparsity in the dataset. Together, these experiments highlight the versatility of regression techniques: Linear Regression excels at predicting continuous outcomes with interpretable trends, while Logistic Regression provides robust and reliable binary classification in practical business scenarios.